Learning to Get Back Up: Fall Recovery for the MuJoCo Ant

CS 839 Programming Assignment · [TODO: your name] · Fall 2026

1. The behavior (self-righting)

Behavior. After a fall, from a random tipped-over pose (on its side or upside down, sometimes dropped from a height while spinning, on flat, bumpy or sloped ground, and possibly shoved), the Ant must get back onto its feet and **stay** standing still.

Why. Fall recovery is a prerequisite for autonomous legged robots deployed in inspection, mining and search and rescue, where a fall otherwise requires a human to walk into a hazardous area to stand the robot up. Learned recovery is an active topic (e.g. FR-Net, 2025) because scripted get-up motions fail on unusual poses and terrain.

Why it is non-trivial for the Ant. The Ant's hips only swing in the torso plane (±30°) and its knees only fold one way, so when inverted its legs point at the sky and the torso is a sphere resting on the ground: it must use leg inertia and brief ground contacts to roll itself over, which is a very different, impulsive contact pattern from walking, and it must then stop and stabilize. Doing nothing never succeeds (0% success; 8% only when dropped from 1–1.5 m, by landing feet-first by luck), and random torques never succeed either (Table 2).

2. The MDP

All of the MDP is in `robot_recovery/ant_recovery.py` (class `AntRecoveryEnv`, parameters in `RecoveryConfig`); the environment used for all final results is `AntRecoveryLowCost-v0`. It subclasses Gymnasium's `MujocoEnv` and uses the unmodified Ant body and actuators from Gymnasium's `ant.xml`, plus a heightfield terrain strip (`robot_recovery/assets/ant_recovery.xml`). Gymnasium 0.29.1 only provides `Ant-v4`, so v4 assets are used (the fallback permitted by the assignment).

Action space. Unchanged: $a \in [-1, 1]^8$, one torque command per hinge (gear 150), clipped and multiplied by a per-episode motor-strength factor.

State (observation) space. 33 dimensions, restricted to quantities a real robot can measure or estimate: gravity direction in the torso frame (3, as an IMU reports it), torso angular velocity (3), torso linear velocity in the torso frame (3), joint angles (8), joint velocities ×0.1 (8), and the previous action (8). Unlike the default Ant observation, it contains the torso orientation in a frame-independent form, and it omits the absolute height and global position, which a real robot does not know.

Is it Markov? No. The underlying simulator state is the full position and velocity of all bodies **plus** the episode's hidden parameters: terrain shape under the robot, friction, payload mass, motor strength, and whether and when a shove will occur. The observation omits torso height above the ground, the terrain, and these parameters, so two identical observations can have different futures (e.g. the same pose on ice vs. on high-friction ground). The previous action is included to restore some short-term information. The process is therefore a POMDP; the policies are memoryless, which is a deliberate choice that matches what a deployed proprioceptive controller would see.

Initial-state distribution (`reset_model`). Each episode:

- **Terrain:** flat ground (p=0.5), bumpy ground (p=0.25; three 4×4 m tiles with smooth bumps up to 6, 10 or 14 cm), or a slope (p=0.25; three pyramids with 5°, 10°, 15° faces, landing 0.6–1.4 m from the apex, facing any direction).
- **Pose:** random heading, then tilted by U(120°, 180°) about a random horizontal axis; joint angles uniform within their limits; placed 2 cm above the highest ground point within 1.2 m.
- **Drop:** with p=0.3 an extra U(0.3, 1.0) m of height and an angular velocity U(−3, 3) rad/s per axis.
- **Domain randomization:** sliding friction ×U(0.8, 1.2), torso mass ×U(0.9, 1.3), motor strength ×U(0.9, 1.1).
- **Disturbance:** with p=0.5 one shove at a random step in [40, 300): a horizontal velocity change of U(0.3, 1.0) m/s in a random direction plus U(−2, 2) rad/s angular velocity per axis.

Reward (dense shaping; **not** the performance metric). With c = torso up-axis · terrain normal, h = torso height above the terrain, ω = torso angular speed:

$$r = \left(\frac{c+1}{2}\right)^2 + 0.5 \cdot \mathrm{clip}\left(\frac{h-0.25}{0.3}, 0, 1\right)\mathbb{1}[c > 0.7] + 0.5 \cdot \mathbb{1}[c > 0.9]e^{-\omega^2/2} - 0.005\ \|a\|^2 - 0.01\ \|a - a_{\mathrm{prev}}\|^2\ .$$

The terms reward being upright, standing tall (the torso only rises above its 0.25 m radius when the legs carry it), and being still once upright, and penalize effort and jerky actions (smoothness matters on real motors). `AntRecovery-v0` uses twice these two cost weights (0.01, 0.02); halving them was the single most important change for PPO (§5).

Episode length and termination. 400 steps of $\Delta t = 0.05$ s (20 s; physics at 100 Hz, 5 sub-steps per action). The default Ant "healthy height" termination is removed, since it would end every episode immediately for a fallen robot. The only termination is a non-finite simulator state (never observed). Episodes end by time-limit truncation; note that CleanRL's SAC treats truncation correctly (bootstraps), while CleanRL's PPO treats it as a terminal state.

3. True performance metric

Success (binary, per episode): the Ant is standing upright and steady for the **entire final 2.5 s** (last 50 steps) of the episode. "Standing upright and steady" at a step means all of:

1. $c \geq 0.9$: torso up-axis within 25.8° of the local terrain normal (finite differences of the heightfield over ±0.15 m at the spawn point; $(0, 0, 1)$ on flat ground and rubble);
2. $h \geq 0.32$ m: torso centre at least 0.32 m above the terrain directly below it (lying on the torso gives ≈0.25 m; relaxed standing ≈0.40 m);
3. $\omega \leq 1$ rad/s: torso angular speed.

We report the **success rate** (mean over episodes). Secondary: **time to recover**, the first time (s) the condition holds for 1 s continuously (20 s if never). The metric captures exactly the goal (ending on its feet, upright and stable) and is interpretable without the reward: "the Ant recovered in 95% of falls". It cannot be satisfied by partial progress the reward pays for (being half-rolled, standing up briefly and falling again, or rocking while upright), and it requires **staying** up after any shove.

4. Learning curves (PPO vs. SAC)

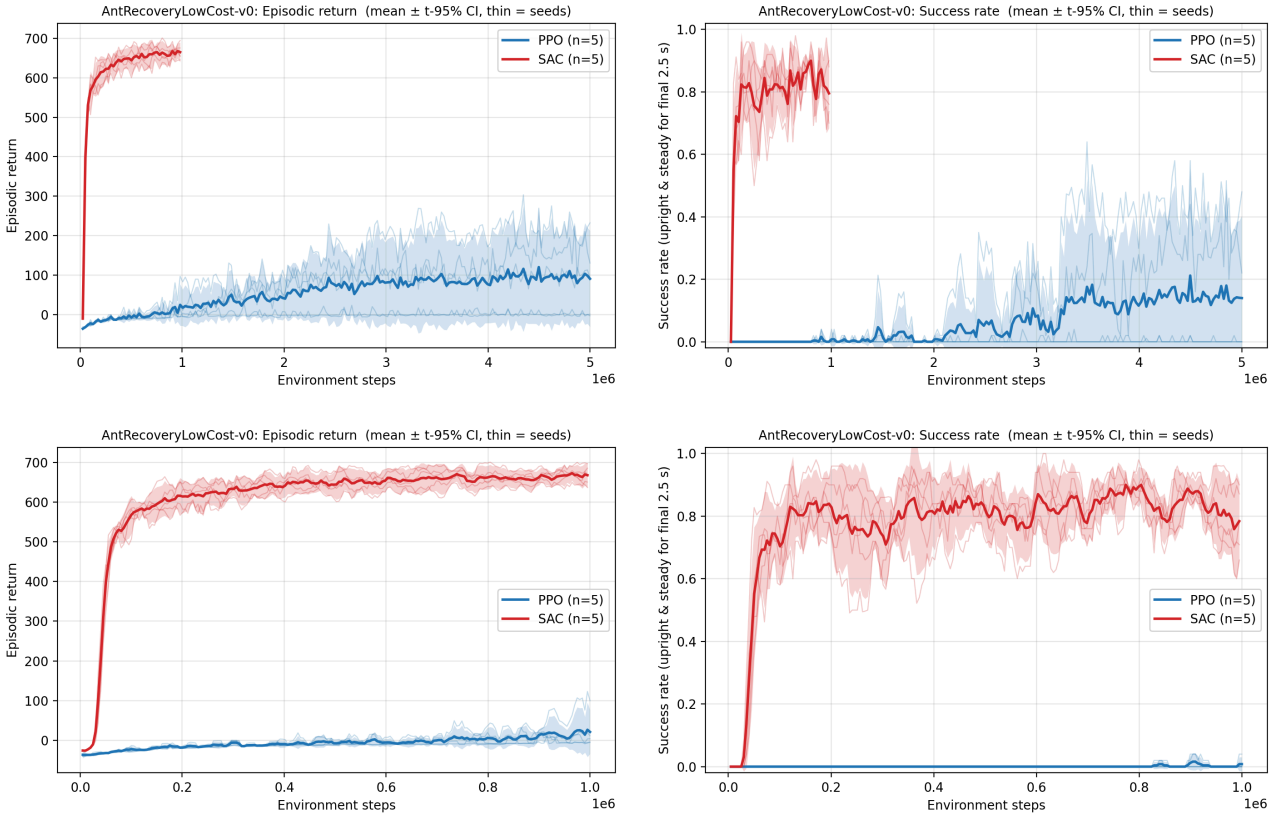


Figure 1: Episodic return (left) and true metric, success rate (right) vs. environment steps on `AntRecoveryLowCost-v0`, full runs (top) and the equal-budget first 1M steps (bottom). Bold: mean over 5 paired seeds (1–5 for both algorithms); band: Student-t 95% interval $\bar{x} \pm t_{0.975,4}\, s/\sqrt{5} = \bar{x} \pm 2.78\, s/\sqrt{5}$; thin lines: individual seeds. SAC was trained for 1M steps, PPO for 5M.

Aggregation. Each run logs one value per finished training episode (stochastic policy) at irregular steps. For each seed we smooth with a trailing 50-episode moving average, then linearly interpolate onto a common grid of 200 evenly spaced environment steps (never extrapolating outside the run), and only then average across seeds (smooth-then-aggregate). The mean and interval are drawn only where all 5 seeds have data. Non-overlapping intervals are not read as significance tests.

Results (mean ± t-95% half-width, per-seed values in parentheses; training episodes):

- **SAC** reaches 76 ± 20% success by 0.25M steps and ends at **80 ± 11%** at 1M (76, 90, 74, 88, 70); return 665 ± 25; mean time to recover 3.7 ± 0.5 s. All 5 seeds learn.
- **PPO** at the same 1M steps: **0.8 ± 2.2%** (0, 4, 0, 0, 0), return 17 ± 46. At 5M steps: **14 ± 26%** (48, 22, 0, 0, 0), return 91 ± 122. Only 2 of 5 seeds ever learn to get up; the other 3 stay on their backs. The wide, symmetric interval describes this bimodal outcome badly, which is why the per-seed curves are shown.
- The training-curve plateau of SAC (≈80%) reflects its stochastic exploration policy: evaluated deterministically, the same five policies succeed in **99%** of training-distribution episodes (Table 2).

5. Comparison and tuning

Which algorithm learned the task better, and on which axis. SAC, on every axis. **Final performance:** 80% (training) / 99% (deterministic evaluation) for SAC vs. 14% / 17% for PPO, despite PPO receiving 5× the data. **Sample efficiency:** at equal environment steps (1M) SAC is at 80% and PPO at 0.8%; SAC passes 50% success within roughly 50k steps, while PPO's best seed needs 2.5–3M steps to reach 20%. **Variance across seeds:** SAC's five seeds end

between 70% and 90% (deterministic: 97–98% averaged over all conditions), whereas PPO is bimodal (two seeds learn, three never do), so PPO's outcome depends more on the seed than on the algorithm's hyperparameters.

Why. (1) **Discovering the flip is rare, and only SAC can reuse it.** Rolling over requires a coordinated burst of leg motion; early on it happens in a small fraction of episodes. SAC stores every such transition in a 1M-step replay buffer and performs one gradient update per environment step, so a rare success is replayed many times. PPO learns only from its latest 2,048-step batch (10 epochs, then discarded), so a lucky flip contributes to a handful of updates and is forgotten. (2) **A cheap local optimum.** Lying still on the back incurs no control cost, and the reward for being partly tilted is small; PPO's Gaussian policy, with no entropy bonus, quickly shrinks its exploration around this "do nothing" behavior (adding an entropy bonus made it worse, not better, in our tuning). SAC's maximum-entropy objective with an automatically tuned temperature keeps exploring until it finds the flip. (3) **Capacity and normalization.** CleanRL's PPO uses 64-unit tanh networks and normalized rewards, while SAC uses 256-unit ReLU networks; we tested 256-unit PPO networks and they did not help, so capacity is not the bottleneck. (4) **Time limits.** CleanRL's PPO treats the 20 s time-limit truncation as a terminal state, so the value of "standing at the end of the episode" is cut off, whereas CleanRL's SAC bootstraps through truncation. This is a smaller effect but biases PPO against the behavior we want late in the episode.

Tuning. SAC used CleanRL defaults throughout. All PPO tuning runs used 5M steps; the job lists are in `chtc/batches/ppotune*.txt`. Success at 5M steps (mean of the last 100 training episodes), mean (per seed):

Reward	Envs × steps	Entropy	Width	Success at 5M
`v0` (costs 0.01/0.02)	1×2048	0	64	0% (0, 0)
`v0`	1×2048	0.01	64	4% (8, 0)
`v0`	16×256	0 / 0.01	64	0% (all four seeds)
low-cost	**1×2048**	**0**	**64**	**35% (46, 23)**, chosen (CleanRL defaults)
low-cost	1×2048	0.01	64	0% (0, 0)
low-cost	16×256	0	64	24% (49, 0)
low-cost	16×256	0.01	64	10% (17, 2)
low-cost	1×2048	0	256	17% (0, 23, 27)
low-cost	1×2048	0.01	256	0% (0, 0, 0)
low-cost	16×256	0	256	20% (59, 0, 0)
low-cost	16×256	0.01	256	8% (0, 25, 0)

Table 1: PPO tuning (28 runs). The final PPO configuration reproduces its tuning runs exactly for seeds 1–2.

What worked: halving the control and action-rate cost weights. With the original weights, lying still on the back is the cheapest behavior: PPO never escaped it in 5M steps, and SAC reached 88% vs. 95% (single pilot seeds) with the lower costs, so the change was adopted for both algorithms. What did not work: an entropy bonus of 0.01, 16 parallel environments with shorter rollouts, and 4× wider (256-unit) networks.

6. Robustness beyond the training distribution

Because the end goal is a real robot, we evaluated every trained policy deterministically in 12 conditions (100 episodes each, identical episode seeds for every policy) and with two reference policies. Each condition changes one factor relative to the training distribution, except "hard", which combines unseen terrain, slopes up to 25°, 1–2 m/s shoves, 0.5–1.5 m drops, friction ×0.4–1.5, payload ×1–1.8 and motors ×0.7–1.1. The rubble field (70 tilted, partly buried bricks and slabs) never appears in training.

Condition	SAC	PPO	Do nothing	Random
Training mix	**99** (99–100)	17 (0–47)	0	0
Flat	**100** (100–100)	16 (0–48)	0	0
Bumpy (≤14 cm)	**100** (100–100)	14 (0–37)	0	0
Slopes 5–15°	**97** (93–100)	14 (0–41)	0	0
Rubble (unseen)	**100** (100–100)	13 (0–42)	0	0
Shove 1–2 m/s	**99** (98–100)	15 (0–43)	0	0
Drop 1–1.5 m, spinning	**99** (99–100)	22 (0–44)	8	0
Friction ×0.3–0.5	**99** (98–100)	7 (0–23)	0	0
Payload ×1.5–2	**99** (99–100)	12 (0–35)	0	0
Motors ×0.6–0.7	**99** (98–100)	11 (0–45)	0	0
One leg dead	**91** (77–98)	1 (0–3)	0	0
Hard (combined)	**92** (88–100)	12 (0–37)	2	0

Table 2: Success (%) of deterministic policies: mean over the 5 seeds (range over seeds), 100 episodes per condition and seed. “Do nothing” only succeeds when a drop happens to land the Ant on its feet.

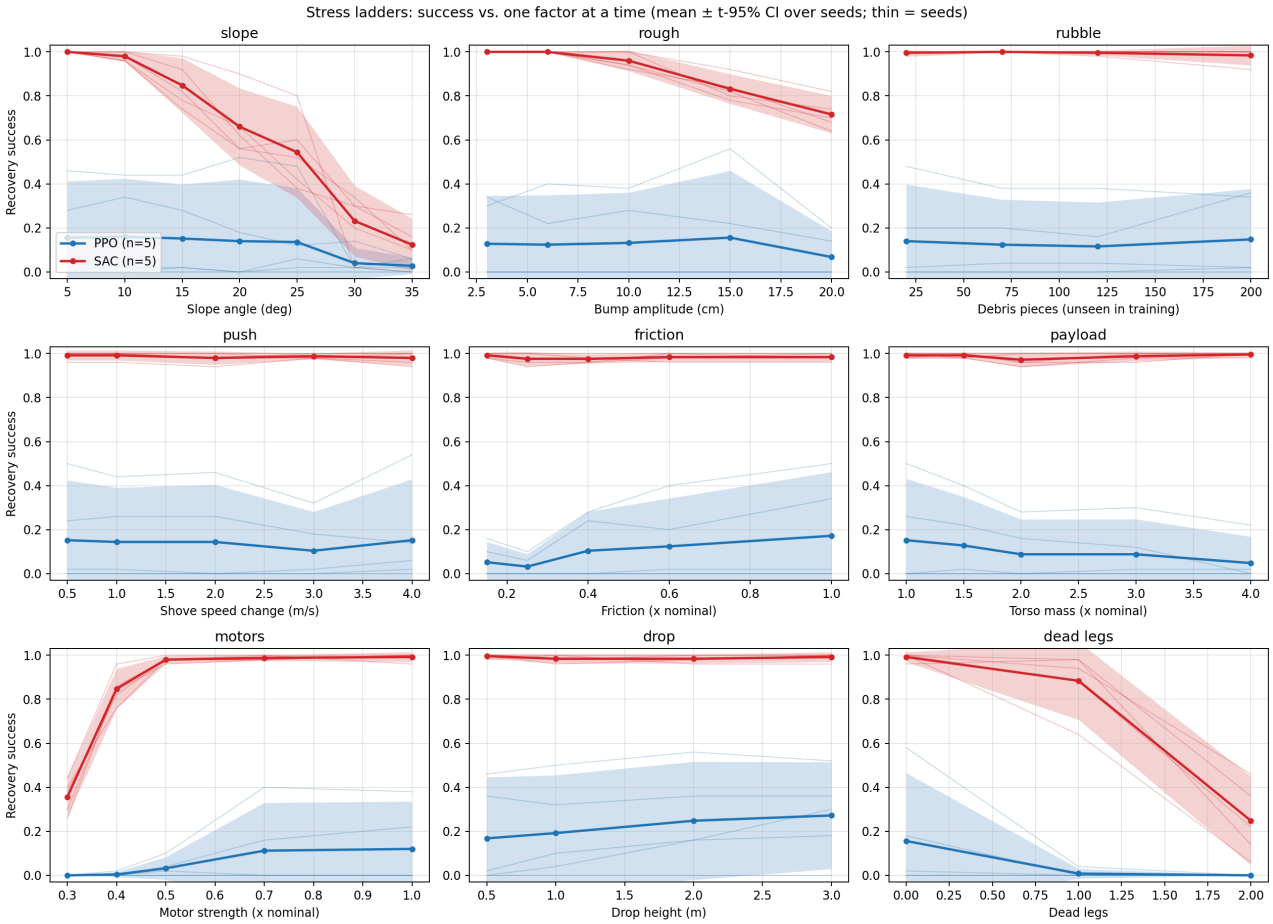


Figure 2: Stress ladders: success vs. one factor at a time (50 episodes per level and seed; mean ± t-95% interval over seeds, thin lines are seeds).

Where recovery breaks (Figure 2; SAC, mean over the 5 seeds). Unaffected (≥97% at every level tested): drops up to 3 m, shoves up to 4 m/s, friction down to ×0.15 (ice), torso payload up to ×4, and rubble fields of up to 200

pieces. Degrading: slope angle (≥98% to 10°, 85% at 15°, the edge of the training range, then 66%, 54%, 23% and 12% at 20–35°), bump amplitude (100% to 6 cm, 96% at 10 cm, i.e. bumps up to 20 cm; 72% at 20 cm, bumps up to 40 cm), motor strength (≥98% down to ×0.5, 85% at ×0.4, 36% at ×0.3) and dead legs (99%, 88% and 25% with 0, 1 and 2 limp legs). Steep slopes and lost actuators, not impacts or payload, are the failure modes to address before moving to a real robot, e.g. by training on steeper terrain and with simulated actuator faults. PPO stays between 0% and 27% at every level.

7. Video

`videos/best_policy_sac_seed4.mp4` (37 s) shows the best policy: SAC seed 4, chosen in advance as the policy with the highest mean success over all 12 evaluation conditions (98.3%). It contains 8 consecutive, unselected training-distribution episodes (episode seeds 10000–10007) with captions giving the terrain and the outcome measured by the true metric; all 8 recover, in 2.1–5.9 s. Each clip ends 1.5 s after the Ant becomes steady (it remains standing until the 20 s mark, which is what the metric checks). The policy rolls off its back by swinging its legs, plants all four feet, and holds still with its torso about 0.5 m high. `videos/highlight_reel_sac_seed4.mp4` (60 s, supplementary) shows one labelled episode per condition at harder settings (20° slope, bumps up to 20 cm, a 200-piece rubble field, a 3 m drop, a 4 m/s shove, ice, ×4 payload, one dead leg), each captioned with the policy's measured success rate at that setting, plus two failures (30° slope, two dead legs). Every success clip is the first episode tried at its setting. Because training ran headless on a cluster, videos were rendered afterwards from the saved policy: `videos/recordvideo_sac_seed4/` holds three full 20 s episodes recorded through Gymnasium's `RecordVideo` wrapper, the mechanism behind CleanRL's `--capture-video`, and the captioned videos use a camera that follows the Ant horizontally at a fixed height so that drops and slopes are visible.

8. Code and reproduction

The code is in the submitted archive: the environment (`robot_recovery/`), the modified XML (`robot_recovery/assets/ant_recovery.xml`), our copies of the two CleanRL scripts (`algos/`), the diff against upstream CleanRL `fe8d8a0` (`patches/cleanrl.diff`), and analysis scripts (`scripts/`). The changes to CleanRL are: importing the environment package; logging `success`, `recovered` and `time_to_recover` when an episode ends; model saving for SAC and saving PPO's observation-normalization statistics with its model; and a PPO `--hidden-size` option (default 64, i.e. unchanged) used only in tuning. No algorithmic code was changed. The final runs are, for seeds $N = 1, \ldots, 5$:

```
python algos/sac_continuous_action.py --env-id AntRecoveryLowCost-v0 --seed N --total-timesteps
1000000 --save-model
python algos/ppo_continuous_action.py --env-id AntRecoveryLowCost-v0 --seed N --total-timesteps
5000000 --save-model
```

`README.md` lists the environment setup, the tuning commands, and the exact commands that produce every figure, table and video. All runs used one CPU core each on UW–Madison's CHTC cluster (no GPU).

9. Use of AI

[TODO: student to review and edit] I used Claude Code (Anthropic's coding agent) throughout this assignment as a pair programmer. With my direction it surveyed related work on legged-robot fall recovery, proposed and implemented the environment (terrain, randomization, reward and metric), made the small changes to the CleanRL scripts, wrote the cluster job scripts, launched and monitored the training runs, and wrote the evaluation, plotting and video scripts and a first draft of this report. I chose the task and its real-world focus, made the experimental decisions (task variants, the training mix, the step budgets and seeds, when to run the final experiments), reviewed the results and videos, and edited the report. I take full responsibility for the submitted work.